Lab 1 Report

Chen Liangkun 23020036007. Lab group: 4.

Lab date: 4th November 2025. Report date: 7th November 2025.

1 Lab Introduction and Team Information

This lab is for installing Ubuntu 20.04 and setting up Git and C++ build tools. Become familiar with basic Shell commands as well as the usage of Git and C++ compilation tools.

2 Individual Work

2.1 Install vitural machine

The first step is to install the virtual machine. We chose Ubuntu 20.04LTS. A ubuntu iso file is required, so we download it from the website. Then we download the VMware and load the iso file to install the system. Because the QingHua mirror is not supported on Ubuntu 20.04, we use the USTC mirror.

To avoid accidents, we should copy the source. List first. And then we can use nano to change the download target to a mirror.

```
sudo cp etc/apt/source.list etc/apt/source.list.bak
sudo vim etc/apt/source.list
sudo apt update
```

```
#USTC Mirror
deb https://mirrors.ustc.edu.cn/ubuntu/ focal main restricted universe
    multiverse
# deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal main restricted universe
    multiverse

deb https://mirrors.ustc.edu.cn/ubuntu/ focal-security main restricted universe
     multiverse
# deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal-security main restricted
    universe multiverse

deb https://mirrors.ustc.edu.cn/ubuntu/ focal-updates main restricted universe
    multiverse
# deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal-updates main restricted
    universe multiverse
```

```
deb https://mirrors.ustc.edu.cn/ubuntu/ focal-backports main restricted
   universe multiverse
# deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal-backports main restricted
   universe multiverse


# deb https://mirrors.ustc.edu.cn/ubuntu/ focal-proposed main restricted
   universe multiverse
# deb-src https://mirrors.ustc.edu.cn/ubuntu/ focal-proposed main restricted
   universe multiverse
```

We now install the tools we need

```
sudo apt install build-essential cmake git
```

2.2 Compile C++

Let us see the file tree for this part:

```
- lab1_demo
        - CMakeLists.txt
        - main.cpp
        - my_lib.hpp
        - my_lib.cpp
```

So we create the CMakeLists.txt to tell CMake how to compile our project. Let's see the CMakeLists.txt below

```
cmake_minimum_required(VERSION 3.10)

project(
  hello_world
  VERSION 1.0
  LANGUAGES CXX)

add_library(mylib my_lib.cpp my_lib.hpp)
add_executable(main main.cpp)
target_link_libraries(main PRIVATE mylib)
```

The first line specifies the minimum CMake version. Then the project tells the project name(hello_world) and its version with language(C++). Next `add_library` will declare and generate a static or dynamic library (static by default):

- Library name: mylib
- Source files: my_lib.cpp (implementation), my_lib.hpp (declaration)

Source files: my_lib.cpp (implementation), my_lib.hpp (declaration) After compilation, a library file will be generated. `add_executable` declares and generates an executable

file named main, with the source file main.cpp. The last line tells the compiler when to compile the main.cpp it needs to link the myli,b either.

```
//main.cpp
#include <iostream>
#include my_lib.hpp

int main() {

    std::cout << Hello world! << std::endl;
    std::cout << my_lib_function() << std::endl;

    return 0;
}
```

This is the main file to print hello world and the return value from `my_lib_function()` from mylib. Now we move to the header file

```
//my_lib.hpp
#pragma once

#include <string>

std::string my_lib_function();
```

`pragma` once prevents the header file from being included multiple times. Here, only one function, `my_lib_function()`, is declared. It tells the compiler that this function is defined (implemented) elsewhere, but can be called from other source files. In other words, it serves as an interface declaration. So for the real implementation

```
//my_lib.cpp
#include <iostream>
#include my_lib.hpp

int main() {

    std::cout << Hello world! << std::endl;
    std::cout << my_lib_function() << std::endl;

    return 0;
}
```

Here is the actual implementation of the function. The compiler will compile this .cpp file into an object file (for example, my_lib.o). CMake will then package it into the library mylib.

CMake recommends using the "Out-of-source" build method, which means placing build files in a separate directory outside the source code directory.[1]

This approach keeps the source directory clean and makes it easier to manage different build configurations. So let's make a "build" directory and get into it. Run the CMake command in the build directory, specifying the source code directory. The source directory is the one containing the CMakeLists.txt file. In other words, it is the parent folder, which

can be specified using two dots. Use the make command to compile and build the project. The command

```
mkdir build && cd build
cmake ..
make
liangkun@ubuntu:~/vnav-personal/lab1/build$ ls
CMakeCache.txt  CMakeFiles  cmake_install.cmake libmylib.a  main  Makefile
```

```
liangkun@ubuntu:~/vnav-personal/lab1/build$ ./main
Hello world!
In library
```

Wow, we have now successfully finished building a C++ project and compiled it!

2.3 Git

First, st we need to connect to GitHub by ssh-key. We first generate the key by the following command.

```
liangkun@ubuntu:~/Desktop$ ssh-keygen -t rsa -C 3140308969@qq.com
Generating a public/private rsa key pair.
Enter file in which to save the key (/home/liangkun/.ssh/id_rsa):
Created directory '/home/liangkun/.ssh'.
Enter passphrase (empty for no passphrase):
Enter the same passphrase again:
Your identification has been saved in /home/liangkun/.ssh/id_rsa
Your public key has been saved in /home/liangkun/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:dTV9JSUjSEkHNPLF21dUz+KOgcM2s8c/PYzripfzG1g 3140308969@qq.com
The key's randomart image is:
+---[RSA 3072]----+
|        .+*=+ *+B|
|         oo+.o *=|
|          o .o. =|
|         o o.....|
|        S B .E.. |
|         . *o+   |
|          ..=.+. |
|          .= ooo.|
|         ...==+..|
+----[SHA256]-----+
```

Then we find the key in the ∼`/.ssh/id_rsa.pub` and copy it to GitHub to create a connection.

Then let's confirm the connection with the command below:

```
liangkun@ubuntu:~$ ssh -T git@github.com
The authenticity of host 'github.com (20.205.243.166)' can't be established.
ECDSA key fingerprint is SHA256:p2QAMXNIC1TJYWeIOttrVc98/R1BUFWu3/LiyKgUfQM.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

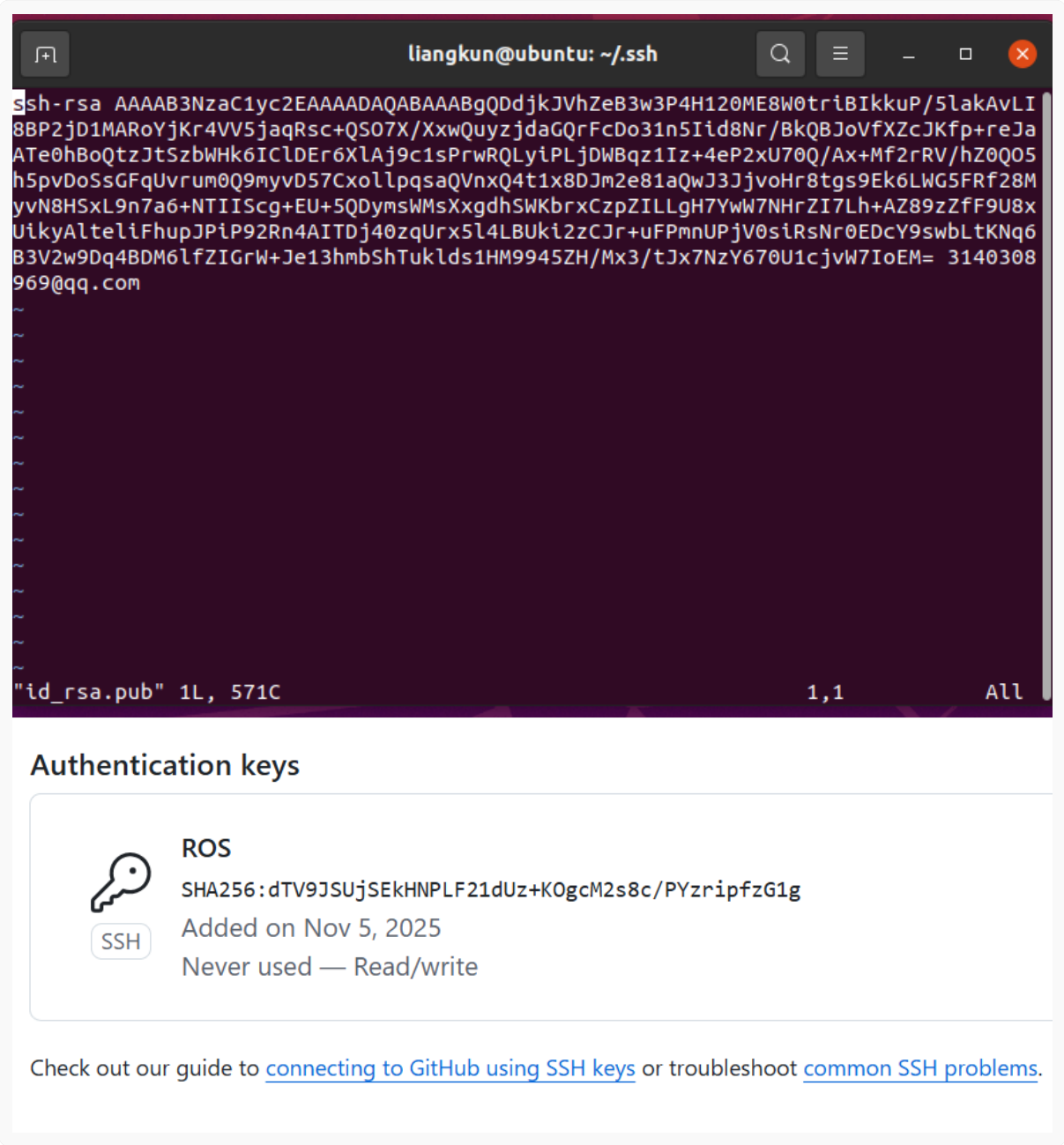

Figure 1: The ssh key

```
Warning: Permanently added 'github.com,20.205.243.166' (ECDSA) to the list of
    known hosts.
Hi KunLiangChen! You've successfully authenticated, but GitHub does not provide
     shell access.
```

Now we can use git to connect github. Next step, we clone the repository with the clone command:

```
liangkun@ubuntu:~$ mkdir ROS_course
liangkun@ubuntu:~$ cd ROS_course/
liangkun@ubuntu:~/ROS_course$ git clone  https://github.com/ouc-vlab-course/
    vnav-codes
Cloning into 'vnav-codes'...
remote: Enumerating objects: 593, done.
remote: Counting objects: 100% (593/593), doneRemotete: Compressing objects:
    100% (534/534), done.
remote: Total 593 (delta 119), reused 2 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (593/593), 2.35 MiB | 2.85 MiB/s, done.
Resolving deltas: 100% (119/119), done.
liangkun@ubuntu:~/ROS_course$ ls
vnav-codes
```

As ls shown, we have already cloned the repository to our computer.

2.4 Sehll

First of course, we need to download the txt file for statistics.

```
liangkun@ubuntu:~/ROS_course/Lab1$ wget  https://raw.githubusercontent.com/
    dlang/dmd/master/druntime/benchmark/extra-files/dante.txt
--2025-11-04 17:24:00--  https://raw.githubusercontent.com/dlang/dmd/master/
    druntime/benchmark/extra-files/dante.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
    185.199.111.133, 185.199.110.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)
    |185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 557042 (544K) [text/plain]
Saving to: dante.txt

dante.txt           100%[===================>] 543.99K  1.40MB/s    in 0.4s

2025-11-04 17:24:01 (1.40 MB/s) - dante.txt saved [557042/557042]
```

Now let's count the following info with the command wc

- This file contains how many lines
- This file contains how many words
- How many lines are not blank

The Linux `wc` command is used to count words.

With the `wc` command, we can calculate the number of bytes, words, or lines in a file. If no file name is specified, or if the given file name is "-", the `wc` command will read data from the standard input device. By default, wc will count the number of lines, words, and bytes in the specified file. Now let's check them

```
liangkun@ubuntu:~/ROS_course/Lab1$ wc dante.txt
 19567  97676 557042 dante.txt
```

So there are 19567 lines and 97676 words. To calculate the lines that are not blank, we introduce the pipe and grep. Grep matches the given pattern. Grep will, by default, display the lines that contain the matching pattern. And we now see regular expression `'^$'`, where `^` represents the beginning of a line and $ represents the end. A line that ends next to the beginning is empty.

Pipe — can connect two commands and take the last command's output as the next command's input, thus we input command

```
liangkun@ubuntu:~/ROS_course/Lab1$ grep -v '^$' dante.txt | wc -l
14338
```

-v is an option of the grep command. It performs a reverse search, printing only the lines that do not match the pattern. So now we print all the lines that are not empty and transfer this text to the wc command and count the number of lines, which is 14338.

Next step, we introduce output redirect. Let's see an introduction frthe om MIT VNAV course.[2] We will now record the required result into exercise1.txt.

```
liangkun@ubuntu:~/ROS_course/Lab1$ echo Total lines: $(wc -l < dante.txt) >
    exercise1.txt
liangkun@ubuntu:~/ROS_course/Lab1$ echo Total words: $(wc -w < dante.txt) >>
    exercise1.txt
liangkun@ubuntu:~/ROS_course/Lab1$ echo Non-empty lines: $(grep -v '^$' dante.
    txt | wc -l) >> exercise1.txt
liangkun@ubuntu:~/ROS_course/Lab1$ cat exercise1.txt
Total lines: 19567
Total words: 97676
Non-empty lines: 14338
```

Note, we use >>to append.

Fortunes will randomly output a proverb or a humorous statement. Using the output redirect introduced upwards, we perform a for loop to record 5 tiFortune'sunes output.

```
liangkun@ubuntu:~/vnav-personal/lab1$ for i in {1..5}; do
>     fortune >> ~/vnav-personal/lab1/fortunes.txt
> done
```

Now let's use cat or vim to check what's in the exercise.txt. We can see in Figure 3 that we gather 5 different sentences. Note that "You will soon forget this" is really something output from fortunes.

Output redirect

We redirect the output of a command to a file. This is useful when we want to save the output of a program without writing specific code.

The common commands that we use and their results are

- `command > output.txt`

The standard output stream will be redirected to the file only, it will not be visible in the terminal. If the file already exists, it gets overwritten.

- `command &> output.txt`

Both the standard output and standard error stream will be redirected to the file only, nothing will be visible in the terminal. If the file already exists, it gets overwritten.

- `command | tee output.txt`

The standard output stream will be copied to the file, it will still be visible in the terminal. If the file already exists, it gets overwritten.

- `command |& tee output.txt`

Both the standard output and standard error streams will be copied to the file while still being visible in the terminal. If the file already exists, it gets overwritten.

Moreover.

If you want to append instead of overwrite you can use the double angle brackets `>>`. With `tee` instead add the flag `-a` (e.g. `tee -a output.txt`).

Figure 2: The introduction to output redirect

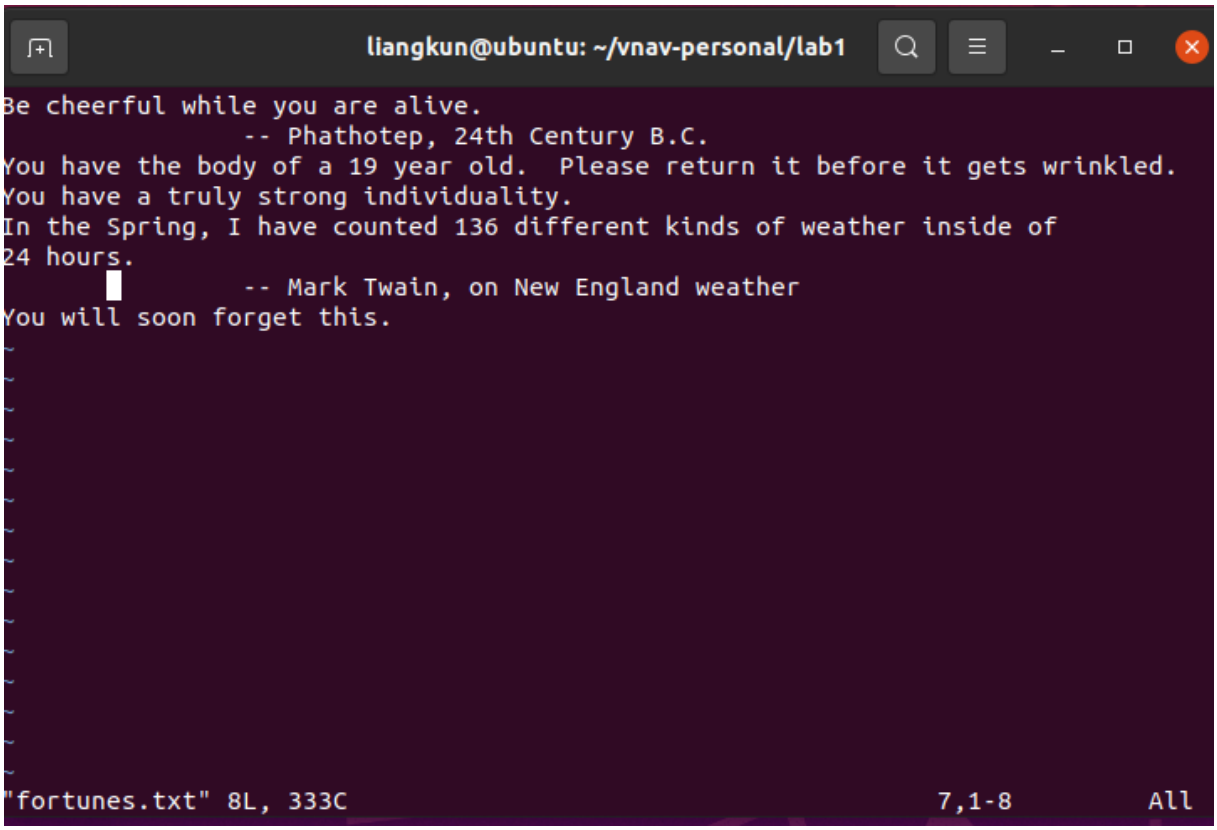

Figure 3: The fortunes.txt

2.5 C++

This section is directed by CPPreference[3].

1. Questions: What are the values of i and j after running the following code? int i = 0, j; j = ++i; j = i++;

   Ans: i = 2, j = 1. ++i add 1 to i first then do the assign operation. So j is 1. i++ do the assignment first then add 1 to i so j is still 1 but i becomes 2.

2. Questions: What does the following code print?

```
int i = 42;
std::string output = (i < 42) ? a : b;
std::cout << output << std::endl;
```

   Ans: i is not less than 42 so the condition is wrong, the triple operator "?" will return b. So it print b

3. Questions: What does the following code print?

```
int i;
int& ri = i;
i = 5;
ri = 10;
std::cout << i <<   << ri << std::endl;
```

Ans: ri is a pointer point to i. We assign ri as 10 actually is assign i to 10. So it will print "10 10".

4. Questions: What does the following code print?

```
int i = 42;
int* j = &i;
*j = *j**j;
std::cout << *j << std::endl;
```

Ans: j is the pointer to i. *j**j = i × i. Therefore it print out $42 \times 42 = 1764$

5. Questions: What does the following code print?

```
int i[4] = {42,24,42,24};
*(i+2) = *(i+1)-i[3];
std::cout << *(i+2) << std::endl;
```

Ans: i is a pointer to the first element of the array. $*(i+2)$ is the value of the third values. $*(i+1)$ is the vlue of the second element and i[3] is point to the last element. So we assign 0 to i

$$2$$

, it print 0.

6. Questions: What does the following code print?

```
void reset(int &i) {
    i = 0;
}

int j = 42;
reset(j);
std::cout << j << std::endl;
```

Ans: Because the parameter of reset is a pointer, so when change the pointer the values of the variable outside the function also will change. Now j is assign to 0 so it print out 0.

7. Questions: What are the differences between int, long, long long, and short?

Ans: As their bytes vary, they have different range. These data will have little different depend on different system and compiler. We take our Ubuntu system as example. sizeof is a powerful tool that can tell the length of a specific data length. So we use followed code

```
#include <stdio.h>
int main()
{
```

```
    short a = 10;
    int b = 100;

    int short_length = sizeof a;
    int int_length = sizeof(b);
    int long_length = sizeof(long);
    int long_long_length = sizeof(long long);

    printf(short=%d, int=%d, long=%d, long long=%d\n, short_length,
int_length, long_length, long_long_length);

    return 0;
}
```

```
liangkun@ubuntu:~/vnav-personal/lab1$ vim lab1_1.cpp
liangkun@ubuntu:~/vnav-personal/lab1$ gcc lab1_1.cpp -o lab1_1
liangkun@ubuntu:~/vnav-personal/lab1$ ./lab1_1
short=2, int=4, long=8, long long=8
```

So as we can see in our machine(linux) long is 8 byte long while some machine(windows) it is 4 bytes. Through these informations we can define their range(The range also influenced by whether it's unsigned or not). But the biggest different can be seen on how many bytes they have in machine.

8. Questions: What are the differences between a float and double? What is the value of i after running the following code snippet? int i; i = 3.14;‘

   Ans: Down to the system actually each type of var will be allocate specific length. So for float it take 4 bytes and double take 8 bytes.

   A prvalue of floating-point type can be converted to a prvalue of any integer type. The fractional part is truncated, that is, the fractional part is discarded.[3] So you will get 3 as i's value.

9. Questions: 3. What are the differences between an unsigned and signed type? What is the value of ‘c‘ in the following code snippet assuming chars are 8-bit?

   unsigned char c = -1;‘

   Ans:To represent the positive and negative numbers, the computer performs a method called two's complement. Signed var will consider such a method, so it can represent both positive and negative numbers. Unsigned var will ignore this and just do the calculation, turn binary to decimal. So it only represents non-negative numbers. for -1 in char. it actually is 11111111. Now we don't perform $-2^7 + 2^6 = -1$ but $2^7 + 2^6 + \ldots 2^0 = 255$. So the values is 255.

   Questions: 4. What will the value of ‘i‘ be after running the following code snippet?

```
int i = 42;
if (i) {
 i = 0;
} else {
```

```
    i = 43;
   }
```

Ans: A prvalue of integral, floating-point, unscoped enumeration, pointer, and pointer-to-member types can be converted to a prvalue of type bool.

The value zero (for integral, floating-point, and unscoped enumeration) and the null pointer and the null pointer-to-member values become false. All other values become true.[3] Therefore condition is true, the value of i is 0.

2.6 C++ extension

In this section, we will implement the RandomVector class. First, we already have the main file and header file. We need to implement the method announced in it. We first give out the implementation code.

```
#include random_vector.h
#include <random>
#include <iomanip>

// TODO: add any include you might require

RandomVector::RandomVector(int size, double max_val) {
  std::random_device rd;
  std::mt19937 generator(rd());
  std::uniform_real_distribution<double> distribution(0,max_val);
  for(int i=0;i<size;i++){
    vect.push_back(distribution(generator));
  }
}

void RandomVector::print(){
  std::cout<<RandomVector:␣;
  for(auto v : vect)
    std::cout << v << ␣;
  std::cout << std::endl;
}

double RandomVector::mean(){
  double sum = 0;
  for (auto v : vect)
    sum += v;
  if (vect.empty()) return 0.0;
  return sum / vect.size();
}

double RandomVector::max(){
  if (vect.empty()) {
        throw std::runtime_error(RandomVector::max()␣called␣on␣empty␣vector);
    }
```

```
  double max = vect[0];
  for (auto v : vect)
    if (v > max) max = v;
  return max;
}

double RandomVector::min(){
  if(vect.empty())
  {
    throw std::runtime_error(RandomVector::min()␣called␣on␣empty␣vector);
  }
  double min = vect[0];
  for(auto v : vect)
    if(v < min) min = v;
  return min;
}

void RandomVector::printHistogram(int bins){
if (vect.empty())
    throw std::runtime_error(printHistogram()␣called␣on␣empty␣vector);

  double min_val = min();
  double max_val = max();
  double range = max_val - min_val;
  if (range <= 0)
    range = 1e-9; // avoid division by zero

  std::vector<int> hist(bins, 0);


  for (auto v : vect) {
    int idx = (int)((v - min_val) / range * bins);
    if (idx == bins) idx--;
    hist[idx]++;
  }


  int max_count = 0;
  for (auto h : hist)
    if (h > max_count)
      max_count = h;


  for (int level = max_count; level > 0; --level) {
    for (int j = 0; j < bins; j++) {
      if (hist[j] >= level)
        std::cout << ***␣;
      else
        std::cout << ␣␣␣␣;
    }
    std::cout << std::endl;
```

```
  }
}
```

First, we build the constructor function. We include a random pack and call it to generate the random values and push them into vector vect. Thus, we finish the RandomVector initialization.

The vector provides several functions, thus we can use them to implement mean, max, and min. For the mean function, we use a for loop to count the sum of vect and divide by vect.size(). We need to avoid dividing by 0, so we set an if to return 0.0 when the size is 0. For the max and min function, we first detect whether vect is empty. Then we set a variable to record the max/min value. We compare elements in the vect and find out the max/min value. Print function, we iterate the vect and print them one by one.

To print out Histogram we still detect whether it's an empty vector or not. Next, we call the max and min functions to find the range of the values in vect. To avoid the range becoming zero(max=min), we set a very tiny value. Now we set a hist vector to store the histogram. We initialize it by setting its length to bins and all assign 0. Next, we calculate the index in hist for each value in vect. Let's do a quick proof.

$$min + i \times box_length \leq v < min + (i+1) \times box_length, \tag{1}$$

$$i \times box_length \leq v - min < (i+1) \times box_length, \tag{2}$$

$$i = \lfloor \frac{v - min}{range} \times bins \rfloor. \tag{3}$$

We need to consider the max value actually belongs to the last bin, so if the index is equal to bins, we minus one. Now we record how many values are in each bin in hist.

We print the histogram from the top to bottom. Therefore, we need the bin that has the most values. We set "level" to represent which line we are printing. If a bin has values higher than the "level" now it should print $*$; else it prints empty. Now we take our program result as an example

```
liangkun@ubuntu:~/vnav-personal/lab1/RandomVector$ vim random_vector.cpp
liangkun@ubuntu:~/vnav-personal/lab1/RandomVector$ g++ -std=c++11 -Wall -
    pedantic -o random_vector main.cpp random_vector.cpp
liangkun@ubuntu:~/vnav-personal/lab1/RandomVector$ ls
main.cpp  random_vector  random_vector.cpp  random_vector.h
liangkun@ubuntu:~/vnav-personal/lab1/RandomVector$ ./random_vector
RandomVector: 0.209168 0.370315 0.791801 0.42821 0.241357 0.112223 0.432664
    0.23705 0.318698 0.67922 0.815383 0.279941 0.277344 0.506064 0.411345
    0.472321 0.257918 0.648872 0.495208 0.802751
Mean: 0.439393
Min: 0.112223
Max: 0.815383
Histogram:
        ***
    *** ***
*** *** ***     ***
*** *** ***     ***
*** *** ***     ***
*** *** *** *** ***
```

As the result shows, the value of hist[0] is 4 now. So when the level is greater than 4, it prints blank. But when the level comes to 4 for the rest time in hist[0] it should print $*$.

3 Discussion

When I install the Linux machine, it actually causes me a lot of trouble. I put my iso file and Linux machine on the D disk, and when I reboot my virtual machine:

> *"I see... only darkness... before me..."*
> *— Arthas Menethil*

Figure 4: Oh come on!

I tried to reinstall the system several times, but it can not solve the problem.

I use AI and search engines, trying a lot of methods and find it may be a problem with GRUB. Seems like my GRUB doesn't know how to find the kernel. Then it took me a lot of time to find how to manually load the kernel and the initial memory disk.

```
set root=(hd0,msdos5)
linux /boot/vmlinuz-5.15.0-139-generic root=/dev/sda5 ro quiet splash
initrd /boot/initrd.img-5.15.0-139-generic
boot
```

Fortunately, this work so me can continue my work.

I have only a little knowledge about C++, so it actually causes me a huge trouble. But I did learn OOP and have some experience coding with C. So with the help of the book, courses, websites, search engines, and AI(I am a trash, sad), learn the basics of C++.

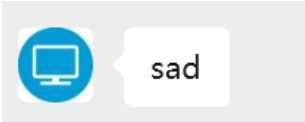

Figure 5: To sad

Actually, I have wanted to learn C++ for a long time, but I don't have such strong determination to learn; this lab actually forced me to learn C++.

4 Conclusions

In this experiment, I install a virtual machine with the Ubuntu system on my computer. Configured the tools it needs to run the various tasks of lab1. Try Git and Shell commands, make a first touch to C++, and try to program with such an exciting language. I even learned something about GRUB as I need to fix my machine. This is actually a meaningful experiment experience.

References

[1] Inc. Kitware and Contributors. Cmake tutorial. `https://cmake.org/cmake/help/latest/guide/tutorial/index.html`, 2025. Accessed: 2025-11-06.

[2] MIT VNAV Lab. Shell Basics: Output Redirection. `https://vnav.mit.edu/labs_2023/lab1/shell.html#output-redirect`, 2023. Accessed: 2025.11.05.

[3] cppreference.com contributors. cppreference.com - C++ Reference. `https://en.cppreference.com/w/`, 2025. Accessed: 2025.11.05.